Game Engine Concepts

Fachhochschule Oberösterreich

Game Scripting Basics

Game Scripting Basics

Introduction

→ **How does a game work?**

- Back to **our game loop**

- Video games are like movies (but more interactive)
  - They have to quickly display a new picture so that things appear like they are moving
- For every picture, your game has to decide what will be on the picture
- You “update” the game state very often
  - For example, 30 times a second (30 fps)
  - You can write a function that Unity will call before drawing each frame

Game Scripting Basics

Introduction

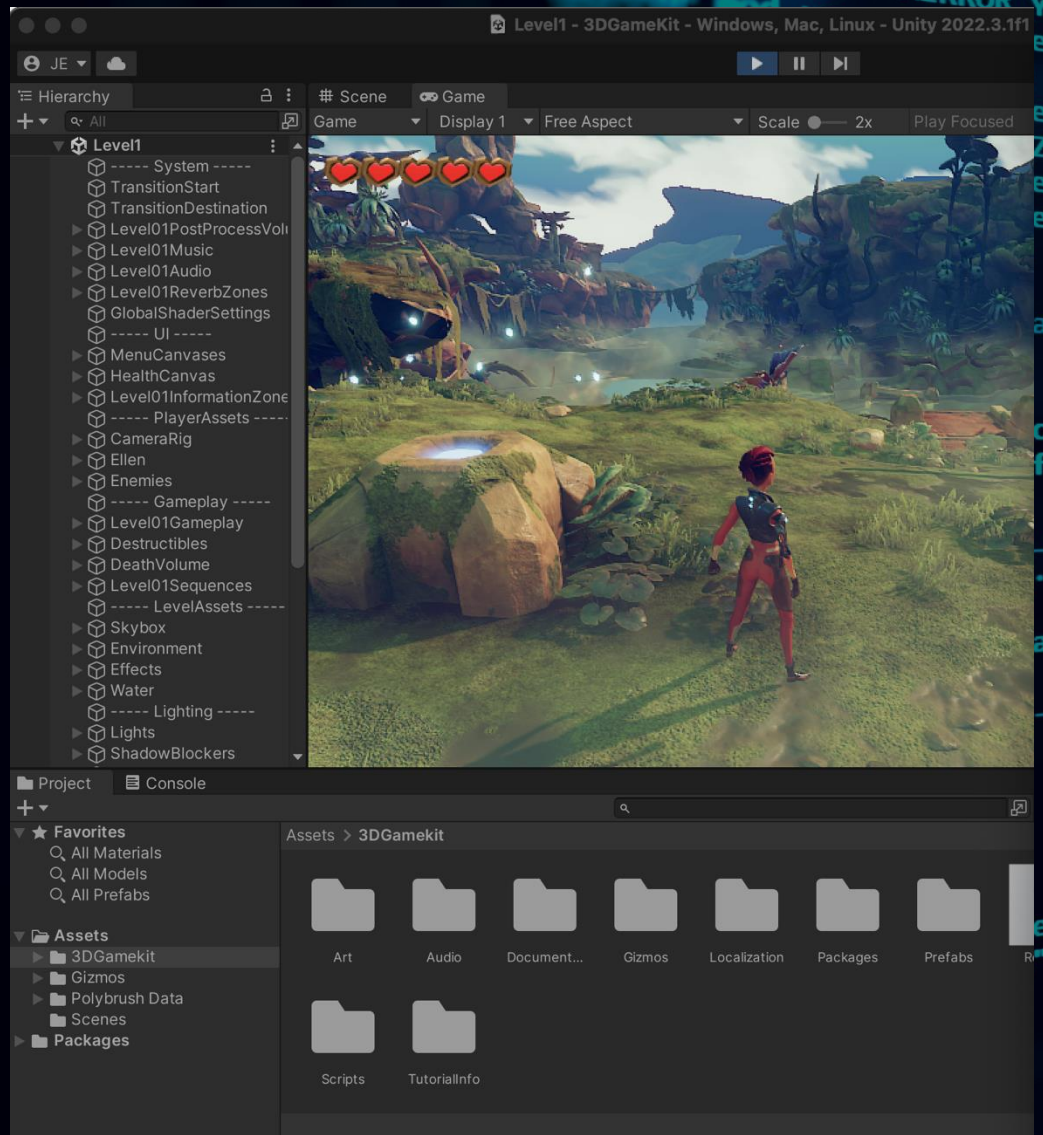
→ **How does a game work?**

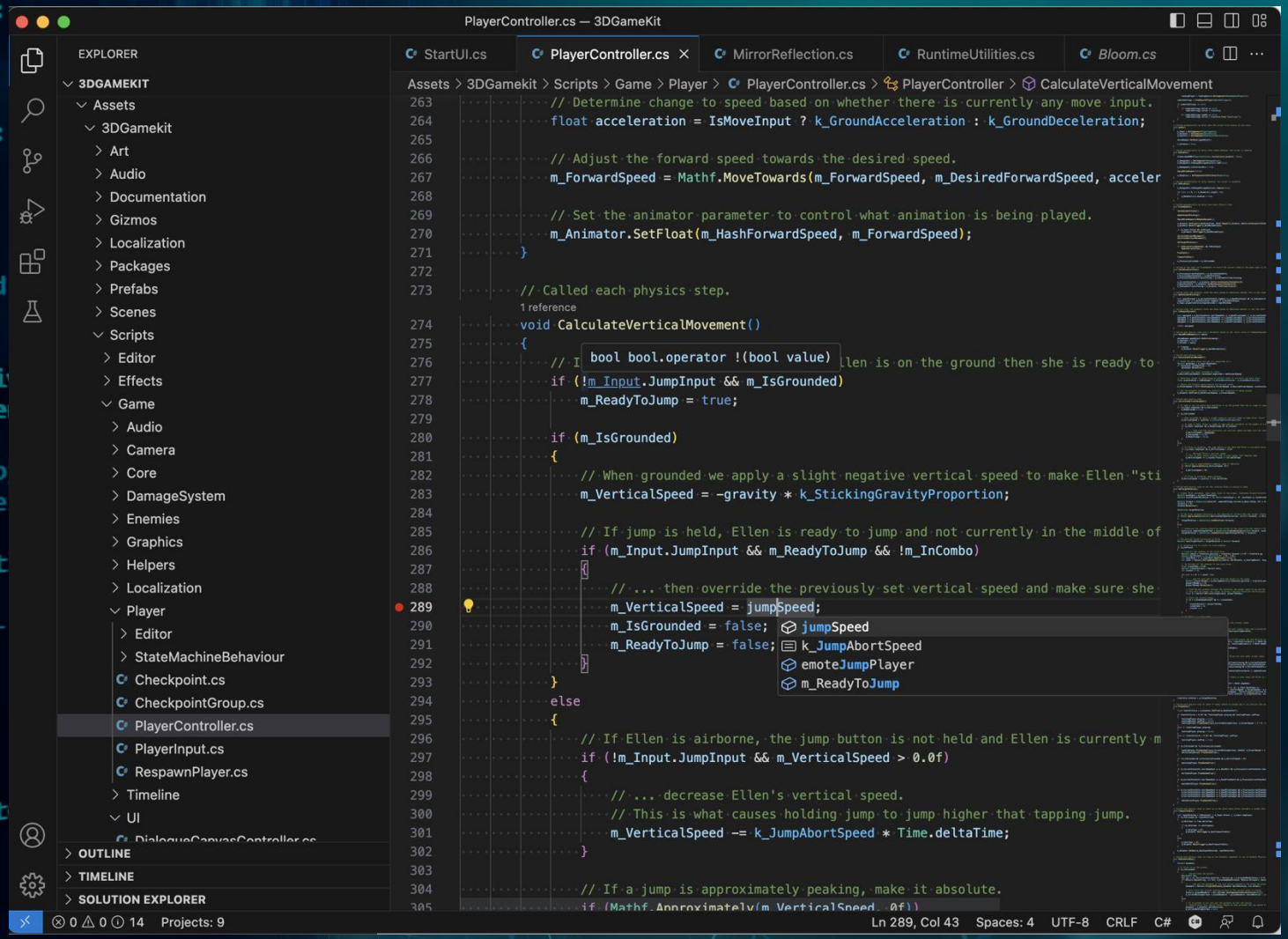
- You **write code**
  - that is able to manipulate the game world
- That **code is executed**
  - at certain points in time

- Let's try to imagine this in **example games**

Game Scripting Basics

Introduction





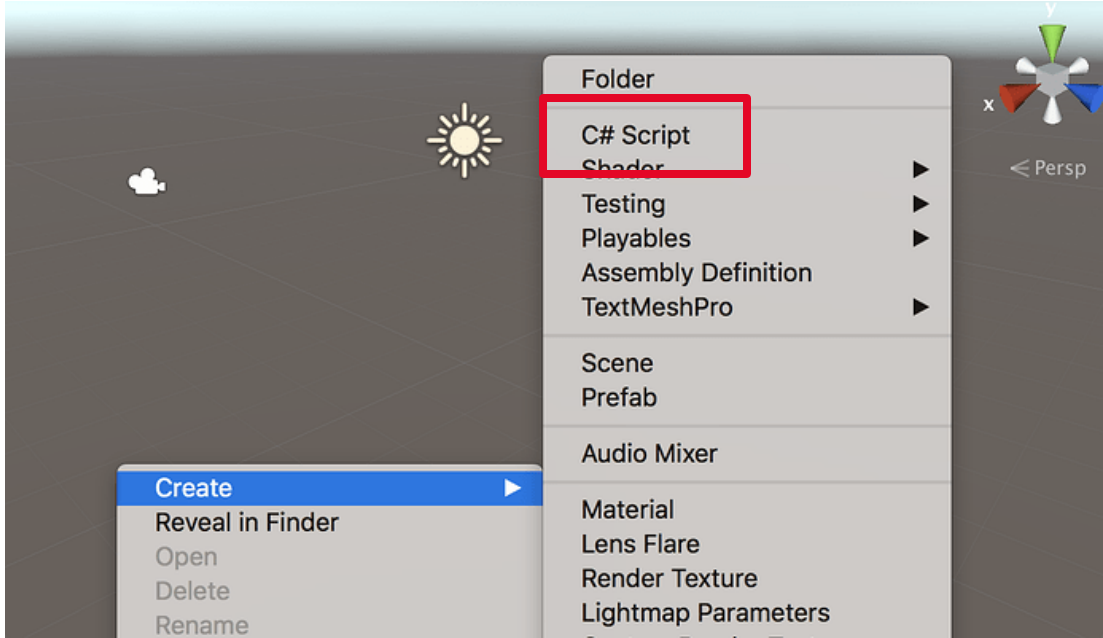
Game Scripting Basics

Introduction

→ **How does a game work?**

- You **write code**
  - Where do you write it into?

- **Create a new C# script (or GDScript, or C++ script …)**
  - This becomes a new component that you can add to game objects just like other components
  - The code in it is not executed if it is not added to a game object



Game Scripting Basics

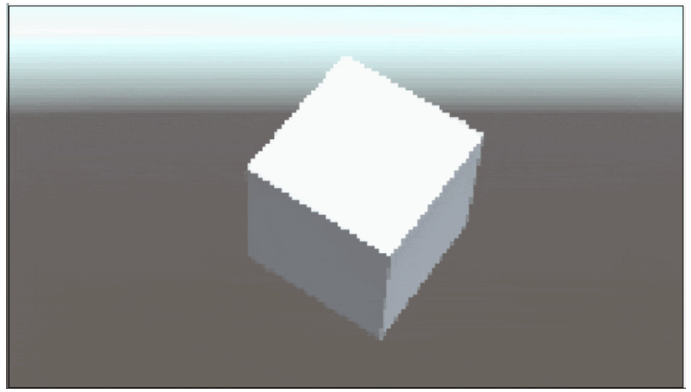
Introduction

→ **How does a game work?**

- You **write code**
  - What can you do with it?

- You can do everything that you could manually do using hierarchy/inspector
  - Move things
  - Rotate things
  - Resize things
  - Check values of things
  - Change colors of things
  - Enable/disable things
  - Create/destroy things

- Scripting Reference: https://docs.unity3d.com/ScriptReference/



Game Scripting Basics

Introduction

→ **How does a game work?**

- That **code is executed**
  - at certain points in time

- **Initialization**
  - Code that runs once when the game starts
- **Animations**
  - Code that runs every frame
- **User input**
  - Code that runs when the user does something
- **In-game events**
  - Code that runs when something interesting happens in the game

Game Scripting Basics

Introduction

→ **How does a game work?**

- Unity provides predefined **functions** that are called when the right time comes

- Common predefined functions in Unity
  - Start
  - Update
  - OnCollisionEnter

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class PlayerMovement : MonoBehaviour {
6
7     // Use this for initialization
8     void Start () {
9
10    }
11
12    // Update is called once per frame
13    void Update () {
14
15    }
16 }
```

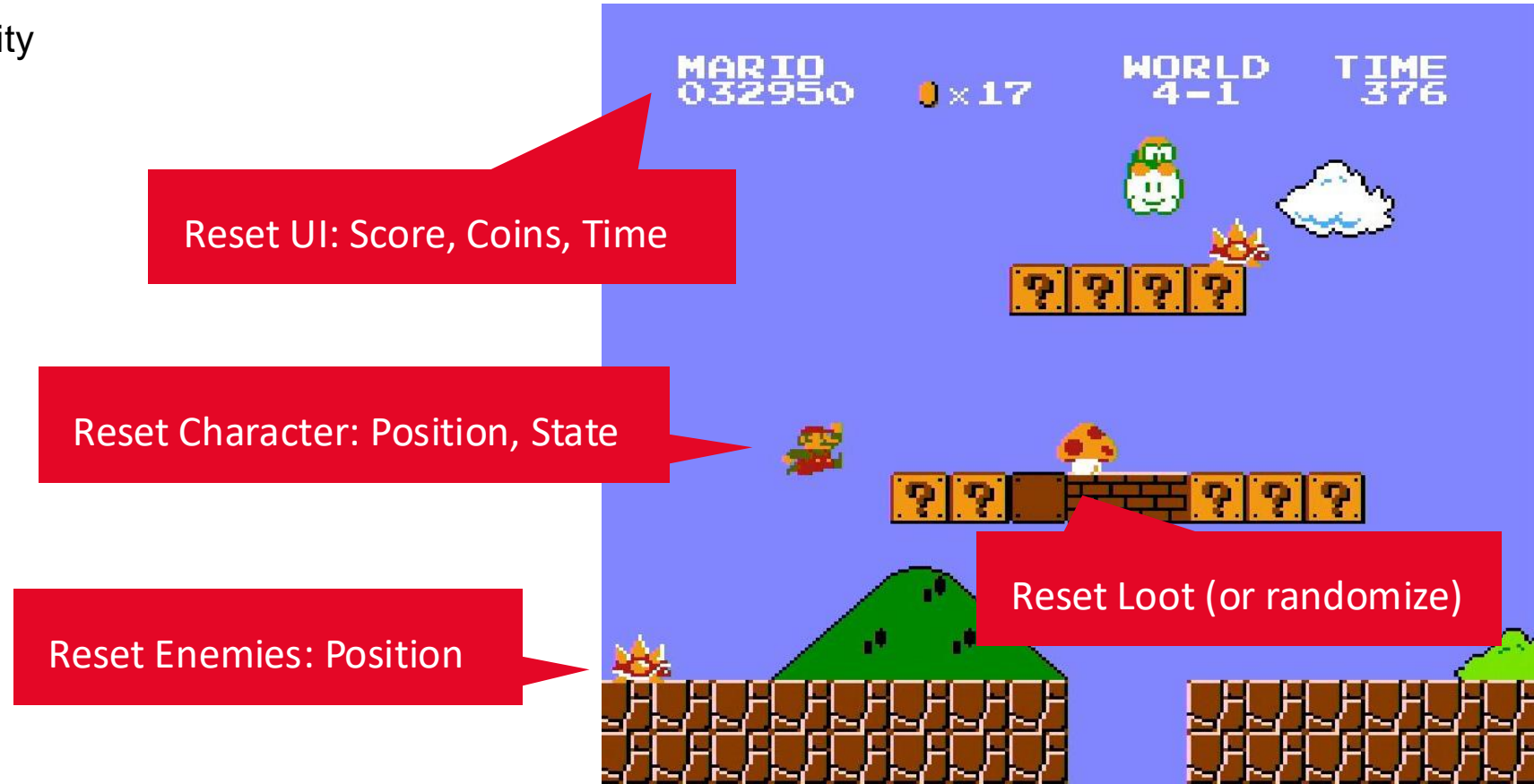
Game Scripting Basics

Introduction

→ **How does a game work?**

- Unity provides predefined **functions** that are called when the right time comes

- Common predefined functions in Unity
  - **Start**
  - Update
  - OnCollisionEnter



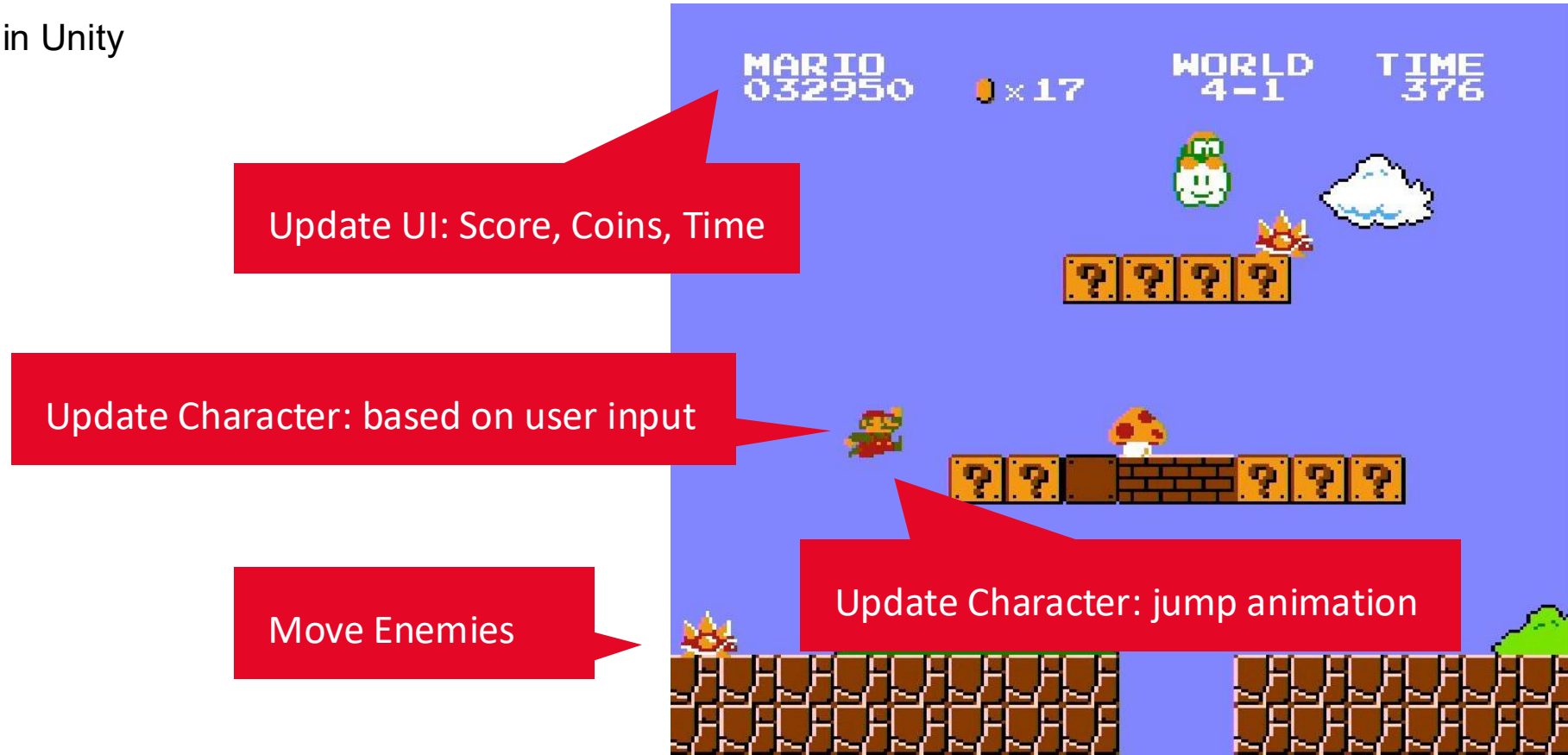
Game Scripting Basics

Introduction

→ **How does a game work?**

- Unity provides predefined **functions** that are called when the right time comes

- Common predefined functions in Unity
  - Start
  - **Update**
  - OnCollisionEnter



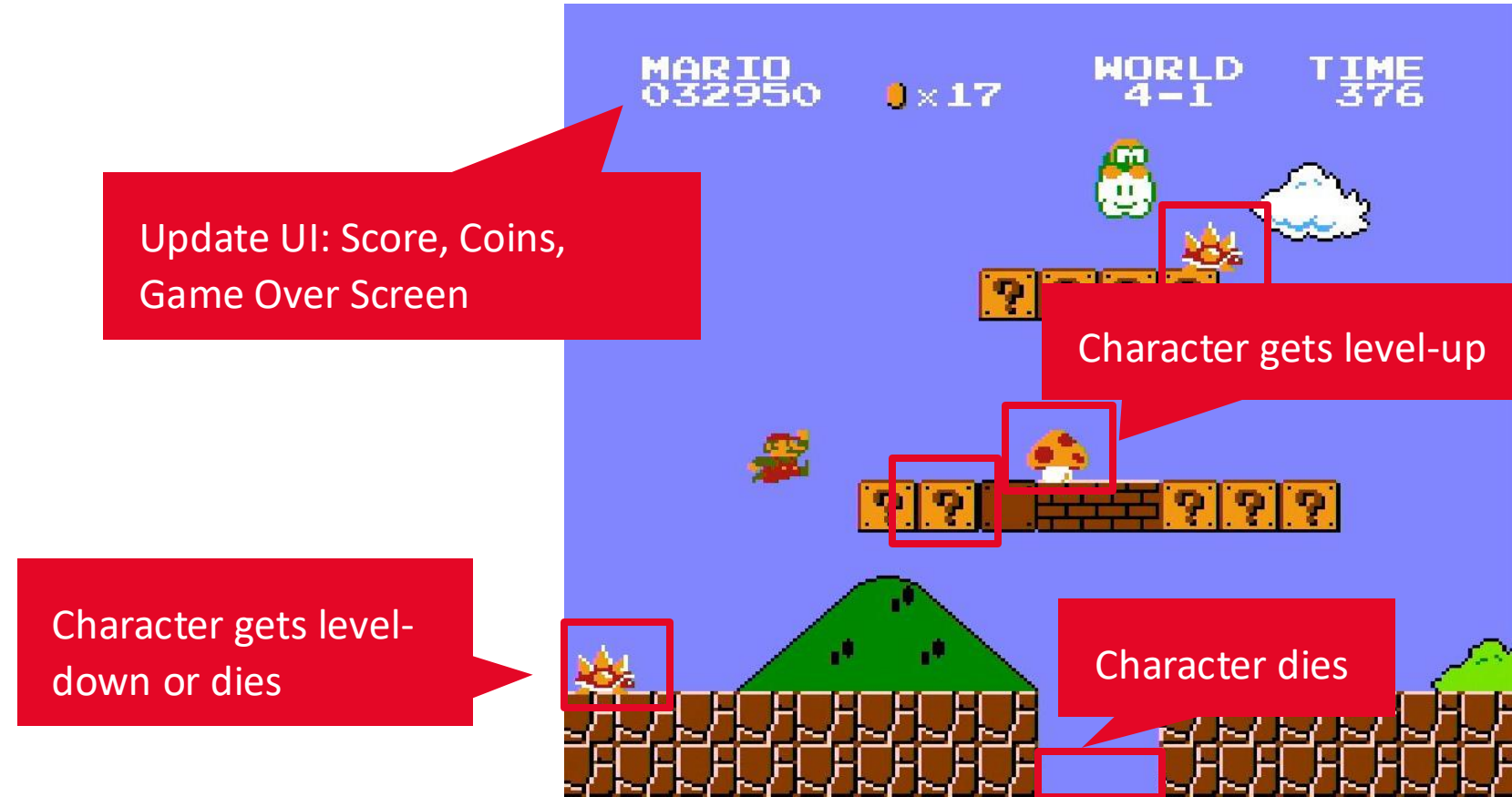
Game Scripting Basics

Introduction

→ **How does a game work?**

- Unity provides predefined **functions** that are called when the right time comes

- Common predefined functions in Unity
  - Start
  - Update
  - **OnCollisionEnter**



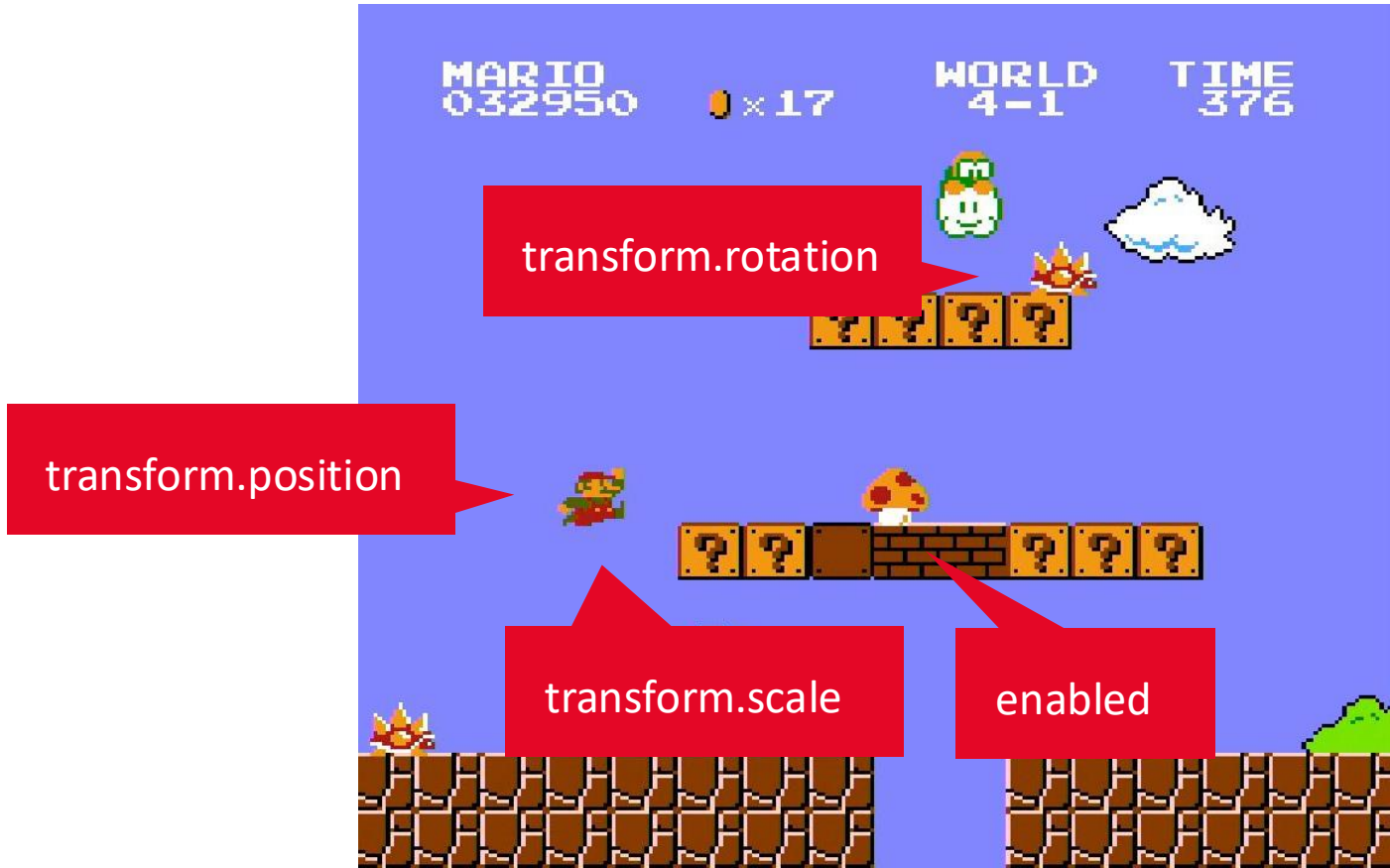
Game Scripting Basics

Introduction

→ **How does a game work?**

- Unity provides predefined **variables** that can be accessed and modified

- Common predefined variables in Unity
  - name
  - gameObject
  - enabled
  - transform
  - renderer



Game Scripting Basics

Introduction

→ **How does a game work?**

- **Scripting** is new for you
  - Learn as you go
  - Don’t try to learn too much, you may confuse yourself
  - Apply and practice everything you read so that it makes sense
  - We have a special lecture next semester: Game Programming

- You will learn details about:
  - **Class** (a collection of variable and function definitions, just a blueprint)
  - **Object** (an actual copy of a class in memory that you can use)
  - **Method** (functions defined in a class)
  - **Field** (variables defined in a class)

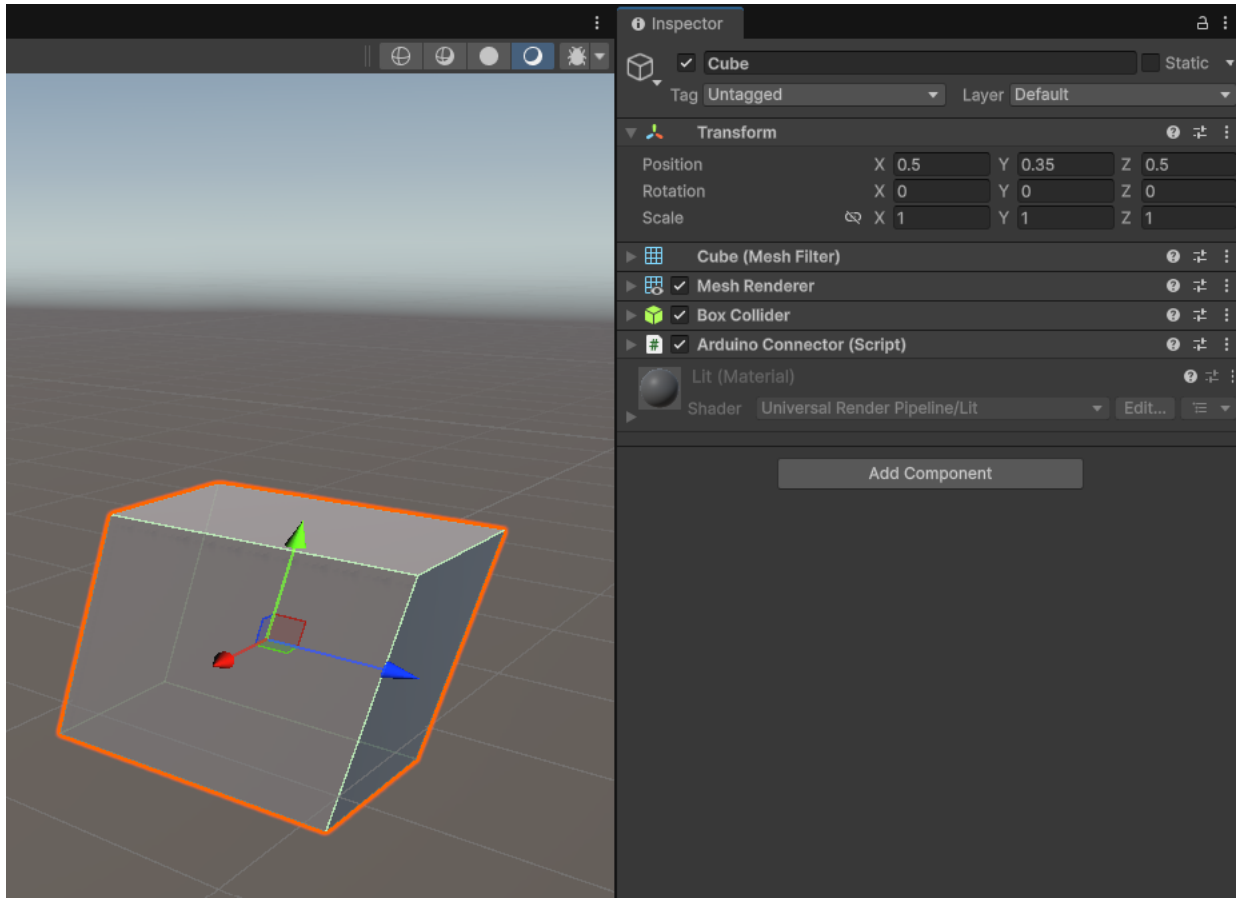
  - Fundamental **data types** (e.g., Vector3, Quaternion)

Game Scripting Basics

Introduction

→ **Game Scripting**

Game Object	**Components**
Cube	Transform Rigidbody MyScript MyOtherScript



Game Scripting Basics

Introduction

→ **Game Scripting**

Game Object

Cube

Components

Transform

Rigidbody

MyScript

MyOtherScript

```
Debug.Log(transform);
Debug.Log(GetComponent<Transform>());
Debug.Log(transform.localPosition);
Debug.Log(transform.localRotation);
Debug.Log(transform.localScale);
Debug.Log(GetComponent<MeshFilter>());
Debug.Log(GetComponent<BoxCollider>());
Debug.Log(GetComponent<Collider>());
Debug.Log(collider);
Debug.Log(GetComponent<MeshRenderer>());
Debug.Log(GetComponent<Renderer>());
Debug.Log(renderer);
Debug.Log(GetComponent<MeshRenderer>().GetInstanceID());
Debug.Log(GetComponent<Renderer>().GetInstanceID());
Debug.Log(renderer.GetInstanceID());
```

```
Cube (UnityEngine.Transform)
UnityEngine.Debug:Log(Object)
Cube (UnityEngine.Transform)
UnityEngine.Debug:Log(Object)
(0.0, 0.0, 0.0)
UnityEngine.Debug:Log(Object)
(0.0, 0.0, 0.0, 1.0)
UnityEngine.Debug:Log(Object)
(1.0, 1.0, 1.0)
UnityEngine.Debug:Log(Object)
Cube (UnityEngine.MeshFilter)
UnityEngine.Debug:Log(Object)
Cube (UnityEngine.BoxCollider)
UnityEngine.Debug:Log(Object)
Cube (UnityEngine.BoxCollider)
UnityEngine.Debug:Log(Object)
Cube (UnityEngine.BoxCollider)
UnityEngine.Debug:Log(Object)
Cube (UnityEngine.MeshRenderer)
UnityEngine.Debug:Log(Object)
Cube (UnityEngine.MeshRenderer)
UnityEngine.Debug:Log(Object)
Cube (UnityEngine.MeshRenderer)
UnityEngine.Debug:Log(Object)
-8246
UnityEngine.Debug:Log(Object)
-8246
UnityEngine.Debug:Log(Object)
-8246
UnityEngine.Debug:Log(Object)
```


Spacehuddle

Game Scripting Basics

Introduction

→ **Game State**

- Examples
  - HP (life)
  - Ammo
  - Inventory items
  - Cool-down time for a skill
  - Experience
  - …

- How can the game state be scripted?
  - Using classes and variables

```
public class MyCharacter : MonoBehaviour {
        public int healthPoints = 100;
}
```

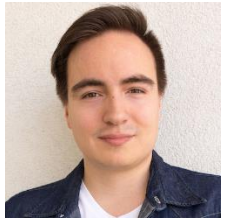
The instance of the class will reflect the state of the GameObject that it is attached to

Game Engine Concepts

FH Oberösterreich
University of Applied Sciences Upper Austria

Digital Media Department

Media Technology & Design
Digital Arts
Interactive Media



Dr. Andreas Riegler
Professor of Interactive Systems
Digital Media Department
E Andreas.Riegler@fh-hagenberg.at